Contexto

Na etapa anterior você generalizou os planos de assinatura, tornou `Conteudo` abstrata e travou com `final` o que não podia mudar. Mas ficou uma dívida desde que o `Podcast` nasceu: a `Playlist` e a `Plataforma` continuam só aceitando `Musica`. Na vida real, você monta uma fila com uma música, um episódio de podcast e outra música, e o app toca tudo em sequência sem perguntar o tipo de cada item.

Além disso, o Sonora precisa ganhar dinheiro também com quem não paga: entram os anúncios, que tocam entre as músicas de quem está no plano gratuito e são cobrados dos anunciantes. Um anúncio não é um conteúdo do acervo, nem um plano, mas tem coisas em comum com os dois: toca como um conteúdo e gera receita como um plano.

Agora você vai resolver essas duas situações com polimorfismo (fazer objetos diferentes responderem, cada um do seu jeito, à mesma mensagem) e interfaces (definir um comportamento que classes sem parentesco algum podem prometer cumprir).

Preparação

Esta atividade continua incremental. Clone o projeto da etapa anterior e trabalhe por cima dele:

- **Copie o projeto da etapa anterior para uma nova pasta** e mantenha o pacote `sonora`.
- Nada do que já funcionava pode quebrar: ao terminar, o projeto continua compilando e rodando.

O que você vai praticar

- Variável polimórfica, tipo estático e tipo dinâmico.
- Coleções de uma superclasse guardando objetos de várias subclasses.
- Quais membros podem ser usados a partir do tipo estático, e quando o `instanceof` com downcasting é necessário (e quando não é).
- Interface: declaração, `implements`, realização de mais de uma interface pela mesma classe.
- Interface como tipo de variável polimórfica.
- Classe abstrata que implementa interface sem implementar os métodos dela.
- Herança entre interfaces e método `default`.

Parte A: o diagrama vem antes do código

Como na etapa anterior, leia as Partes B a D inteiras antes de programar e atualize o diagrama de classes do Sonora com o que você pretende construir:

1. As associações de `Playlist` e `Plataforma` agora apontando para `Conteudo` em vez de `Musica`, com os adornos revisados.
2. As interfaces com o estereótipo `<<interface>>`, a herança entre interfaces e o relacionamento de realização (linha tracejada com triângulo vazado) de cada classe que implementa uma interface.
3. A nova classe `Anuncio` e a sua associação com `Plataforma`.

Se alguma decisão mudar durante a implementação, corrija o diagrama. Ele precisa terminar coerente com o código.

Parte B: polimorfismo com a hierarquia que você já tem

B1. Playlist e Plataforma aceitam qualquer conteúdo

1. Troque o `ArrayList<Musica>` da `Playlist` por `ArrayList<Conteudo>`. Ajuste os parâmetros e retornos dos métodos que recebiam ou devolviam `Musica`.
2. Faça o mesmo com o acervo da `Plataforma`: ela passa a cadastrar músicas e podcasts na mesma coleção.
3. `getDuracaoTotalSegundos()` e `reproduzirTudo()` da `Playlist` devem funcionar com uma playlist misturando músicas e podcasts, sem nenhum `if` perguntando o tipo do item. Repare: quem decide qual `getCreditos()` é executado dentro do `reproduzir()` é o tipo dinâmico de cada objeto.

B2. A receita da plataforma

Assim como no exemplo da aula, em que a empresa quer saber quanto paga de salário a todos os funcionários, o Sonora quer saber quanto recebe por mês.

1. Crie em `Plataforma` o método `calcularReceitaMensal()`, que soma a mensalidade do plano de todos os usuários cadastrados.
2. Um único laço, sem `instanceof` e sem cast. Se você se pegar escrevendo um laço por tipo de plano, volte e pense no que o método abstrato da etapa anterior já resolve.

B3. O que o tipo estático deixa fazer

1. No `App`, declare `Conteudo c = new Podcast(...)` e escreva em comentário qual é o tipo estático e qual é o tipo dinâmico de `c`.
2. Deixe comentada a linha `// c.getApresentador();` com uma explicação de por que ela não compila, mesmo o objeto sendo um podcast.
3. Crie em `Plataforma` o método `listarEpisodiosDe(String apresentador)`, que percorre o acervo e imprime os podcasts daquele apresentador. Aqui o `instanceof` com downcasting é legítimo: o método precisa de um membro que só existe em `Podcast`.

Parte C: anúncios e interfaces

C1. A classe Anuncio

Crie a classe `Anuncio`. Ela não herda de `Conteudo` (um anúncio não faz parte do acervo, não entra em playlist e não aparece em busca) nem de `Plano`.

- `anunciante` e `titulo` (texto, obrigatórios).
- `duracaoSegundos` (maior que zero) e `valorPorMilImpressoes` (maior que zero), validados com `IllegalArgumentException`.
- `impressoes` (inteiro, começa em zero, com getter e sem setter).
- `reproduzir()`: soma uma impressão e imprime algo como `[Anuncio] Cafe da Serra - oferecimento de Torrefacao Serrana`.
- `calcularValorVeiculacao()`: devolve `impressoes / 1000.0 * valorPorMilImpressoes`.

Na `Plataforma`, adicione uma coleção de anúncios e um método para cadastrá-los.

C2. O problema

A receita mensal agora precisa somar os planos e os anúncios. O estagiário já tentou, e o resultado foi este:

```
public double calcularReceitaMensal() {
```

```
    ArrayList<Object> cobrancas = new ArrayList<>();
    for (Usuario u : usuarios) {
        cobrancas.add(u.getPlano());
    }
    for (Anuncio a : anuncios) {
        cobrancas.add(a);
    }

    double total = 0;
    for (Object o : cobrancas) {
        if (o instanceof Plano) {
            total += ((Plano) o).calcularMensalidade();
        } else if (o instanceof Anuncio) {
            total += ((Anuncio) o).calcularValorVeiculacao();
        }
    }
    return total;
}
```

Funciona, mas é o mesmo cheiro do exemplo das despesas e investimentos visto em aula: a lista é de `Object`, cada novo tipo que gerar receita vai exigir mais um `else if`, e esquecer um deles faz o valor sumir em silêncio. Não implemente assim. `Plano` e `Anuncio` não têm parentesco, então herança não resolve. Interface resolve.

C3. A interface Faturavel

1. Crie a interface `Faturavel` com o método `double calcularMensalidade();`.
2. Faça `Plano` implementar `Faturavel` e apague de `Plano` a declaração abstrata de `calcularMensalidade()`. Como `Plano` é abstrata, ela não precisa implementar o método da interface: a obrigação desce para as subclasses concretas, que já o implementam. Compile e confirme que o `resumo()` continua funcionando.
3. Faça `Anuncio` implementar `Faturavel`. O método `calcularValorVeiculacao()` passa a ser a implementação de `calcularMensalidade()` (o que o anunciante paga no mês).

C4. As interfaces Cronometravel e Reproduzivel

Conteúdos e anúncios também têm algo em comum: os dois têm duração e os dois tocam. Crie duas interfaces, uma herdando da outra:

```
public interface Cronometravel {

    int getDuracaoSegundos();

    default String getDuracaoFormatada() {
        int total = getDuracaoSegundos();
        int minutos = total / 60;
        int segundos = total % 60;
        if (segundos < 10) {
            return minutos + ":0" + segundos;
        }
        return minutos + ":" + segundos;
    }
}

public interface Reproduzivel extends Cronometravel {

    void reproduzir();
}
```

1. Faça `Conteudo` implementar `Reproduzivel`. Repare que ela já tem os dois métodos exigidos, inclusive o `reproduzir()` final da etapa anterior.
2. Faça `Anuncio` implementar `Faturavel` e `Reproduzivel` ao mesmo tempo.
3. Observe o método `default`: `Musica`, `Podcast` e `Anuncio` ganham `getDuracaoFormatada()` sem escrever uma linha. Como interfaces não têm atributos, ele só pode usar o que vem de outros métodos, como `getDuracaoSegundos()`.
4. Use `@Override` em todas as implementações de métodos de interface.

Parte D: integre no Sonora

D1. Receita com interface

1. Reescreva `calcularReceitaMensal()` para montar um `ArrayList<Faturavel>` com os planos de todos os usuários e todos os anúncios, e somar tudo em um único laço.
2. Nenhum `instanceof`, nenhum cast.

D2. Tocar uma playlist para um usuário

Crie em `Plataforma` o método `tocarPlaylist(Usuario ouvinte, Playlist playlist)`:

1. Monte uma fila do tipo `ArrayList<Reproduzivel>` com os itens da playlist.
2. Se o plano do ouvinte tem anúncios e existe pelo menos um anúncio cadastrado, intercale um anúncio a cada dois itens da playlist, em rodízio (primeiro anúncio, segundo anúncio, e assim por diante, voltando ao primeiro quando acabarem).
3. Percorra a fila imprimindo a duração formatada e chamando `reproduzir()` de cada item.
4. A decisão de colocar ou não anúncios vem de `temAnuncios()`, não de perguntar o tipo do plano. Dentro deste método, nada de `instanceof`.

Repare no efeito: tocar playlists para usuários do plano gratuito aumenta as impressões dos anúncios, e isso aparece na receita mensal.

D3. Menu e demonstração

1. Adicione ao menu do App as opções para cadastrar um anúncio, adicionar um podcast a uma playlist, tocar uma playlist para um usuário, listar os episódios de um apresentador e exibir a receita mensal da plataforma.
2. Na demonstração, mostre a receita mensal antes e depois de tocar uma playlist para um usuário do plano gratuito.
3. Trate todas as entradas inválidas com exceções. O menu nunca pode quebrar.